SCALE FOR PROJECT CODEXION (/PROJECTS/CODEXION)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42heilbronn.de:vogsphere/intra-uuid-4274cdf3-44de
```

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify with the person (or the group) evaluated the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.

- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's GiT repository.

- Double-check that the GiT repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you and make you evaluate something other than the content of the official repository.

- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.

- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject prior to starting the defence.

- Use the flags available on this scale to signal an empty repository, non-functioning program, a norm error, cheating etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.

- Remember that for the duration of the defence, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explicit the reasons with the evaluated student and make sure both of you are okay with this.

- You must also verify the absence of memory leaks. Any memory allocated on the heap must

be properly freed before the end of execution.
You are allowed to use any of the different tools available on the computer, such as leaks, valgrind, or e_fence. In case of memory leaks, tick the appropriate flag.

Attachments

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/191107/en.subject.pdf)

Preliminaries

Basics

- This project must be coded in C89, compiling with:
  -Wall -Wextra -Werror -pthread
  without any warnings.
- Any compilation error, segmentation fault, data race, deadlock, interleaved log line, or memory leak means 0 to the project.
- The program must take exactly the following arguments, in order: number_of_coders, time_to_burnout, time_to_compile, time_to_debug, time_to_refactor, number_of_compiles_required, dongle_cooldown, scheduler.
- Each coder must be represented by a thread (using pthreads_create).
- The scheduler must be either "fifo" or "edf".
- On some slow hardware, precise timing may vary slightly; discuss borderline cases honestly before counting them as failures.
- Verify compilation: make
- The binary must be named "codexion".

Yes No

Global

- Check if there is any global mutable variable used to manage the shared resources (dongles), the scheduling policy, or the logging.
- If you find such a global state, the evaluation stops here.
  You can go on and check the code, but do not grade the exercises.

Yes No

README.md file

Does the repository contain a README.md file at its root, and does it include all of the following?

- The first line is italicized and formatted exactly as: *This project has been created as part of the 42 curriculum by <login1>[, <login2> , <login3>[...]]*.
- A "Description" section explaining the project's purpose and providing a brief overview.
- An "Instructions" section with relevant details about compilation, installation, and/or execution.
- A "Resources" section listing references (documentation, tutorials, etc.) and explaining how AI was used, specifying for which tasks and which parts of the project.
- A "Blocking cases handled" section describing all the concurrency issues addressed in the solution.
- A "Thread synchronization mechanisms" section explaining the specific threading primitives used.

Yes No

Codexion

Codexion testing — Easy

- Do not test with more than 200 coders.
- Do not test with timing values under 60ms.
- Verify no coder burns out and all complete required compiles.

Yes No

Codexion testing — Less easy

- Validate burnout edge cases.
- Verify correct logging and timing tolerances.
- Check no dongle duplication, correct state transitions.

Yes No

Codexion testing — Medium

- Verify cooldown behavior.
- Test scheduler differences (edf vs fifo).
- Validate refactoring timing and log serialization.

Yes No

Codexion recode — EDF tie-breaker

- Modify EDF to prefer higher coder_id on equal deadlines.
- Validate via contention tests.

Yes No

Ratings

Don’t forget to check the flag corresponding to the defense

Ok | Outstanding project

Empty work | Incomplete work | Invalid compilation | Norme | Cheat

Concerning situation | Leaks | Forbidden function | Can’t support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of pro (https://profile.intra.42